

# Fundamentos de redes neuronales artificiales

## Perceptron multicapa

Emiliano Churruca

Laboratorio de Investigación y Desarrollo Experimental en  
Computación (LIDEC)

**Universidad Nacional de Hurlingham**

(Material adaptado de Juan Santos)

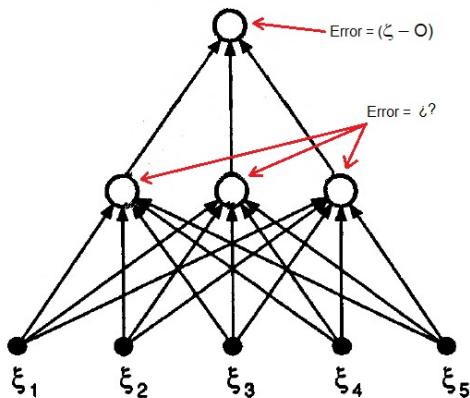
# Redes neuronales artificiales

## Perceptron multicapa

- Aprender en un perceptron simple es responder a la pregunta ¿cómo actualizo los pesos cuando la salida es distinta a la salida esperada?
- Se toma alguna proporción del error y con eso se corrigen los pesos de la conexiones que unen directamente la entrada con las salidas.
- Pero ¿cómo hacerlo en una arquitectura como la que sigue?

# Redes neuronales artificiales

## Perceptron multicapa



# Redes neuronales artificiales

## Perceptron multicapa

- En base al error en la unidad de salida podría corregir los pesos que unen la capa del medio con la unidad de salida, pero ahora ¿cuál es el error en las unidades de la capa del medio?
- Esto es, qué estado de activación deberían tener (deseado) las unidades de la capa del medio para poder establecer un error y de acuerdo a eso corregir los pesos de las conexiones que unen las entradas con las unidades de las capas del medio?

# Redes neuronales artificiales

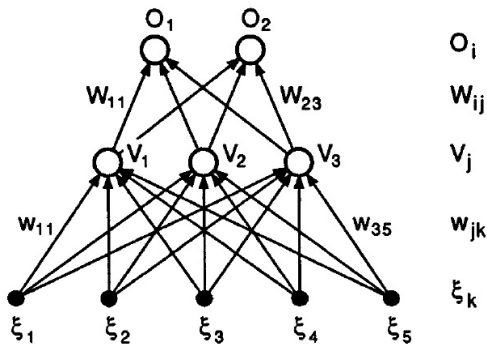
## Perceptron multicapa

- ✧ En 1986, Rumelhart, Hinton, y Williams con su trabajo de PDP (Procesamiento paralelo distributivo) hicieron un importante aporte en este sentido.
- ✧ Hubo varios investigadores que trabajaron en la misma dirección tales como Le Cun (1985), Parker (1985), Werbos (1974), entre otros.

# Redes neuronales artificiales

## Perceptron multicapa

Consideremos una red multicapa con 2 capas.



# Redes neuronales artificiales

## Perceptron multicapa

- ✚ Llamaremos a la capa del medio como **capa oculta**.
- ✚ Las conexiones entre los nodos de entrada y la capa oculta los vamos a notar con  $w$  minúscula, mientras que los pesos de las conexiones entre la capa oculta y la de salida con  $W$  mayúscula.
- ✚ El estado de activación de las unidades en la capa oculta lo notaremos con  $V$  mayúscula.

# Redes neuronales artificiales

## Perceptron multicapa

- El índice  $i$  refiere a las unidades de salida,  $j$  a las unidades ocultas y  $k$  a las unidades de entrada.
- El supraíndice  $\mu$  indica el número de ejemplo (o patrón) y varía entre 1 y  $p$ , donde  $p$  es la cantidad de ejemplos en el conjunto de entrenamiento.
- El subíndice  $k$  indica el elemento del ejemplo y varía entre 1 y  $N$ , donde  $N$  es la dimensión de cada ejemplo.



# Redes neuronales artificiales

## Perceptron multicapa

- Presentado un ejemplo  $\mu$  a la red, cada unidad en la capa oculta alcanza un estado de excitación  $h_j^\mu$  dado por:

$$h_j^\mu = \sum_k w_{jk} \xi_k^\mu$$

lo cual produce un estado de activación de las unidades de la capa oculta  $V_j^\mu$ :

$$V_j^\mu = g(h_j^\mu) = g\left(\sum_k w_{jk} \xi_k^\mu\right)$$

# Redes neuronales artificiales

## Perceptron multicapa

- Cada unidad de salida alcanza un estado de excitación  $h_i^\mu$  dado por:

$$\begin{aligned} h_i^\mu &= \sum_j W_{ij} V_j^\mu = \\ &= \sum_j W_{ij} g\left(\sum_k w_{jk} \xi_k^\mu\right) \end{aligned}$$

lo cual produce un estado de activación de las unidades de salida  $O_i^\mu$ :

$$\begin{aligned} O_i^\mu &= g(h_i^\mu) = g\left(\sum_j W_{ij} V_j^\mu\right) = \\ &= g\left(\sum_j W_{ij} g\left(\sum_k w_{jk} \xi_k^\mu\right)\right) \end{aligned}$$

# Redes neuronales artificiales

## Perceptron multicapa

- Si consideramos la función de costo que ya habíamos visto para el perceptron simple

$$E(w) = \frac{1}{2} \sum_{\mu, i} (\zeta_i^\mu - O_i^\mu)^2,$$

podemos escribirla ahora como

$$E(w) = \frac{1}{2} \sum_{\mu, i} (\zeta_i^\mu - g(\sum_j W_{ij} g(\sum_k w_{jk} \zeta_k^\mu)))^2.$$

# Redes neuronales artificiales

## Perceptron multicapa

- Como se puede observar,  $E()$  depende de  $W_{ij}$  y de  $w_{jk}$  y además es derivable.
- Ahora sí podemos obtener una forma de actualizar los pesos de las conexiones  $w_{jk}$  entre las unidades de entrada y las unidades de la capa oculta.
- **No hace falta conocer explícitamente** un estado de activación deseado para cada una de las unidades en la capa oculta.

# Redes neuronales artificiales

## Perceptron multicapa

- La táctica que implica este enfoque es 'retropropagar' el error  $(\zeta_i^\mu - O_i^\mu)$  de cada unidad de la capa de salida hacia las capas inferiores, en este caso, la capa oculta.
- A partir de esto, se podrán actualizar los pesos de las conexiones  $w_{jk}$ .

# Redes neuronales artificiales

## Perceptron multicapa

✚ Recordemos la expresión del costo:

$$E(w) = \frac{1}{2} \sum_{\mu} \sum_i (\zeta_i^{\mu} - O_i^{\mu})^2$$

$$E(w) = \frac{1}{2} \sum_{\mu} \sum_i (\zeta_i^{\mu} - g(\sum_j W_{ij} V_j^{\mu}))^2$$

# Redes neuronales artificiales

## Perceptron multicapa

- Nuevamente, usando la técnica del gradiente descendente, podemos primero calcular la actualización de los pesos de las conexiones  $W_{ij}$  como

$$\Delta W_{ij} = -\eta \frac{\partial E}{\partial W_{ij}} =$$

$$\Delta W_{ij} = \eta \sum_{\mu} (\zeta_i^{\mu} - O_i^{\mu}) g'(h_i^{\mu}) V_j^{\mu} =$$

$$\Delta W_{ij} = \eta \sum_{\mu} \delta_i^{\mu} V_j^{\mu}$$

donde  $\delta_i^{\mu}$  queda definida como  $\delta_i^{\mu} = (\zeta_i^{\mu} - O_i^{\mu}) g'(h_i^{\mu})$ .

# Redes neuronales artificiales

## Perceptron multicapa

- ✚ Fijarse que la ecuación de  $\Delta W_{ij}$  es muy similar a la del perceptron simple, pero en este caso  $V$  juega el rol que ocupaba  $\xi$ .



# Redes neuronales artificiales

## Perceptron multicapa

✚ En el caso  $\Delta w_{jk}$  usamos la  $\frac{\partial E}{\partial w_{jk}}$

$$\Delta w_{jk} = -\eta \frac{\partial E}{\partial w_{jk}} =$$

$$\Delta w_{jk} = -\eta \sum_{\mu} \frac{\partial E}{\partial V_j^{\mu}} \frac{\partial V_j^{\mu}}{\partial w_{jk}} =$$

$$\Delta w_{jk} = \eta \sum_{\mu i} (\zeta_i^{\mu} - O_i^{\mu}) g'(h_i^{\mu}) W_{ij} g'(h_j^{\mu}) \xi_k^{\mu}$$

# Redes neuronales artificiales

## Perceptron multicapa

✦ Como ya habíamos definido

$$\delta_i^\mu = (\zeta_i^\mu - O_i^\mu)g'(h_i^\mu),$$

podemos re-escribir  $\Delta w_{jk}$  como

$$\Delta w_{jk} = \eta \sum_{\mu i} \delta_i^\mu W_{ij} g'(h_j^\mu) \xi_k^\mu.$$

# Redes neuronales artificiales

## Perceptron multicapa

- Así como para escribir  $\Delta W_i$  definimos  $\delta_i$ , podemos hacer lo mismo con  $\Delta w_{jk}$ .

De este modo

$$\Delta w_{jk} = \eta \sum_{\mu} \delta_j^{\mu} \xi_k^{\mu},$$

donde

$$\delta_j^{\mu} = g'(h_j^{\mu}) \sum_i W_{ij} \delta_i^{\mu}.$$

# Redes neuronales artificiales

## Perceptron multicapa

- Si prestamos especial atención a esta ecuación

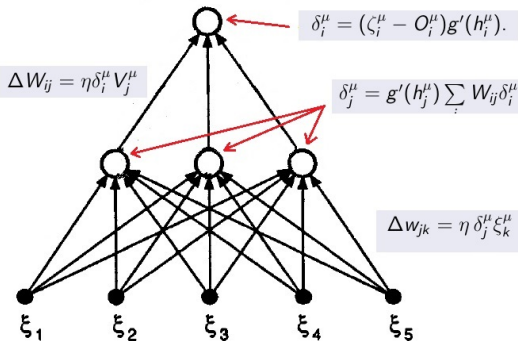
$$\delta_j^\mu = g'(h_j^\mu) \sum_i W_{ij} \delta_i^\mu$$

observamos que permite propagar  $\delta_i^\mu$  (el error en el capa de salida) a  $\delta_j^\mu$  (el error en la capa oculta) mediante los pesos  $W_{ij}$ .

- Es decir, esta ecuación permite propagar el error hacia atrás y de ahí su nombre de 'retropropagación' (backpropagation).

# Redes neuronales artificiales

## Perceptron multicapa



# Redes neuronales artificiales

## Introducción - Aprendizaje en el perceptron

Si, en vez de una capa oculta, tuvieramos más de una capa entre la entrada y la salida, la ecuación general para la actualización de pesos entre dos capas consecutivas  $m - 1$  y  $m$ , donde las unidades en la capa  $m - 1$  están indexadas por  $j$  y en la capa  $m$  por  $i$ , es :

$$\Delta W_{ij} = \eta \sum_{\mu} \delta_i^{\mu} V_j^{\mu}$$

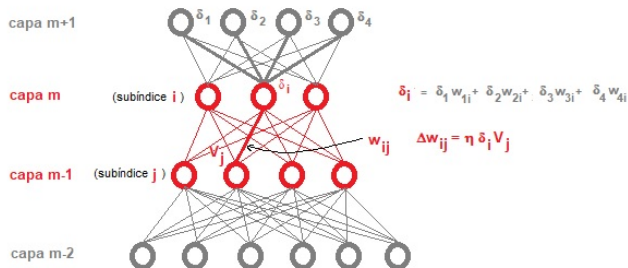
donde  $V_j^{\mu}$  son el estado de activación de las unidades en la capa  $m - 1$ .

Para una entrada  $\mu$  dada, la ecuación es más simple aún:

$$\Delta W_{ij} = \eta \delta_i^{\mu} V_j^{\mu}$$

# Redes neuronales artificiales

## Perceptron multicapa



# Redes neuronales artificiales

## Aprendizaje en el perceptron

- ✧ Fijarse que, la actualización de los pesos entre la capa  $m-1$  y  $m$  (donde  $m-1$  y  $m$  son capas consecutivas) depende de  $\delta_i^\mu$  el cual es propio para cada unidad de la capa  $m$  y de  $V_j^\mu$  el cual es propio de cada unidad de la capa  $m-1$ .
- ✧ Esta característica es muy apropiada para calcular en forma paralela todos los pesos de las conexiones de la red (una vez que sea hayan retropropagado el error desde la salida hasta la capa siguiente de la entrada).



# Redes neuronales artificiales

## Secuencia del aprendizaje

La actualización de los pesos de las conexiones en una red puede realizarse de dos formas:

- incremental, ó
- en lotes.

# Redes neuronales artificiales

## Aprendizaje incremental

- En la forma incremental, cada vez que una entrada es presentada a la red, se propaga dicha entrada hasta obtener los estados de activación de la capa de salida.
- Luego, se retropropaga el error obteniendo los  $\delta$  para cada unidad de cada capa.
- Finalmente se actualiza los pesos de todas las conexiones.
- Esto se repite para cada entrada  $\mu$ .

# Redes neuronales artificiales

## Aprendizaje incremental

- Una vez que se han presentado todas las entradas del conjunto de entrenamiento, se ha completado una 'época'.
- El orden con que se presentan las entradas a la red se recomienda que sea al azar.

# Redes neuronales artificiales

## Aprendizaje en lote

- En la forma en lote, cada vez que una entrada es presentada a la red, se propaga dicha entrada hasta obtener los estados de activación de la capa de salida.
- Luego, se retropropaga el error obteniendo los  $\delta$  para cada unidad de cada capa.
- Finalmente se acumula, para cada conexión, el  $\Delta$  correspondiente.
- Esto se repite un número de veces igual al tamaño del lote.

# Redes neuronales artificiales

## Aprendizaje en lote

- Una vez que se han presentado todas las entradas del lote, se actualizan los pesos de las conexiones sumando el acumulado obtenido.
- El orden con que se presentan las entradas a la red durante el lote no altera el cálculo de los acumulados.
- Una vez que se han presentado todas las entradas del conjunto de entrenamiento, se ha completado una 'época'.

# Redes neuronales artificiales

## Algoritmo. Notación.

- ✚ Consideremos una red con varias capas ocultas cuyo índice de capa es  $m$  y varía entre 0 y  $M$ .
- ✚ La capa 0 es la capa de entrada.
- ✚ La capa  $M$  es la capa de salida.
- ✚ El *error* será calculado entre el valor deseado  $\zeta_i^\mu$  y el estado de activación de la unidad  $i$  de salida cuando el ejemplo  $\xi^\mu$  es presentado en la capa 0 de la red (entrada).
- ✚ El aprendizaje será incremental.

# Redes neuronales artificiales

## Algoritmo. Notación.

- El estado de activación de cada unidad  $i$  en cada capa  $m$  será

$$V_i^m = g\left(\sum_j w_{ij}^m V_j^{m-1}\right)$$

donde  $w_{ij}^m$  son los pesos de las conexiones entre la capa  $m - 1$  y  $m$ .

- El  $\delta$  en cada unidad  $i$  de la capa de salida será

$$\delta_i^M = g'(h_i^M)(\zeta_i^M - V_i^M)$$

- El  $\delta$  en cada unidad  $j$  de la capa  $m - 1$  será

$$\delta_j^{m-1} = g'(h_j^{m-1}) \sum_i w_{ij}^m \delta_i^m.$$

# Redes neuronales artificiales

## Algoritmo

1. Inicializar el conjunto de pesos en valores 'pequeños' al azar.
2. Tomar un ejemplo  $\xi^\mu$  al azar del conjunto de entrenamiento y aplicarlo a la capa 0:  
 $V_k^0 = \xi_k^\mu$  para todo  $k$ .
3. Propagar la entrada hasta a capa de salida  
 $V_i^m = g(h_i^m) = g(\sum_j w_{ij}^m V_j^{m-1})$  para todo  $m$  desde 1 hasta  $M$ .
4. Calcular  $\delta$  para la capa de salida  
 $\delta_i^M = g'(h_i^M)(\zeta_i^\mu - V_i^M)$
5. Retropropagar  $\delta_i^M$  a la capa  $m - 1$   
 $\delta_j^{m-1} = g'(h_j^{m-1}) \sum_i w_{ij}^m \delta_i^m$  para todo  $m$  entre  $M$  y 2
6. Actualizar los pesos de las conexiones de acuerdo  
 $w_{ij}^{nuevo^m} = w_{ij}^{viejo^m} + \Delta w_{ij}^m$   
donde  $\Delta w_{ij}^m = \eta \delta_i^m V_j^{m-1}$
7. Calcular el *error*. Si *error* > *COTA*, ir a 2.



# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

¿Qué tenemos hasta ahora?

- ✚ Olvidemonos por un instante de las neuronas, los pesos sinápticos, etc y concentremos nuestra atención en el método.
- ✚ Tenemos un conjunto de ejemplos y cada uno de ellos tiene una salida asociada (salida deseada).
- ✚ Con esos datos pretendemos aprender la relación que existe entre los ejemplos y sus salidas asociadas.
- ✚ Una vez que la hayamos aprendido, para cada entrada que le presentemos, nos dirá la salida que debería tener.

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- Y esto, no sólo cuando presentemos entradas que son parte del conjunto de ejemplos de entrenamiento, sino que, en principio, para cualquier entrada nueva que se nos ocurra.
- La dependencia entre las entradas y salidas puede ser no lineal, no necesita que las salidas(\*) ni las entradas estén en un rango de valores, no hay limitación en la dimensión de la entrada ni de la salida.

(\*)dependiendo de la función de activación de las unidades de la capa de salida.

# Redes neuronales artificiales

Consideraciones acerca del método de retropropagación.

✈ ¡Maravilloso!

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- ✚ Para obtener la red necesitamos "sólo" saber los pesos de las conexiones entre la neuronas.
- ✚ Pero, dichos pesos están en los reales, de modo que el espacio de posibles valores de los pesos es infinito.
- ✚ Es decir, es como buscar un átomo en el universo.

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- ✚ Sin embargo, la regla de actualización basada en la técnica del gradiente nos presta una invaluable ayuda.
- ✚ Nos permite recorrer ese espacio de valores de los pesos de una manera guiada, marcando un camino que nos conduzca al valor de la solución.

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- ✧ Si fueran 2 los pesos de las conexiones que tenemos que hallar, nosotros podríamos representar a la función  $E(w)$  en un gráfico 3D.
- ✧ Veríamos algo similar a lo ocurre en una cadena montañosa, con picos y valles y lo que estamos buscando es el valle más profundo donde el valor de la función de costo  $E(w)$  será mínimo.

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- ✧ El método del gradiente descendente nos da una dirección en la cual lograremos navegar esa superficie bajando cada vez el costo.
- ✧ Esa dirección es  $\Delta W$ .

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- ✚ El problema que tiene el método del gradiente descendente es que la información que dispone es local.
- ✚ Esto es, porque la dirección de descenso es, justamente, la derivada de  $E(W)$  respecto a cada uno de los pesos de las conexiones en el punto obtenido hasta ese momento.



# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- ✚ Y eso es lo que da como resultado que el método nos conduce a mínimos locales.
- ✚ Y por la propia naturaleza del método, cuando se llega a un mínimo local no hay dirección posible que nos conduzca a seguir bajando.

# Redes neuronales artificiales

Consideraciones acerca del método de retropropagación.

✈ ¡Cáspitas! Era muy 'maravilloso' el método...

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

- Un problema que suele aparecer es el siguiente.
- El error que se retropropaga depende de  $(\zeta_i^\mu - O_i^\mu)$ .
- Cuando el algoritmo se acerca al mínimo global de  $E()$ , esta diferencia se acerca a 0.
- Esto produce que el  $\Delta W$  que depende del error retropropagado se hace muy pequeño y el descenso de  $E()$  entra en una zona plana sin casi modificarse a lo largo del tiempo.

# Redes neuronales artificiales

## Consideraciones acerca del método de retropropagación.

✚ Una alternativa es definir una función de costo que evite esto.

✚ Definamos  $E()$  como

$$E(w) = \sum_{i\mu} \left( \frac{1}{2} (1 + \zeta_i^\mu) \log \frac{1 + \zeta_i^\mu}{1 + O_i^\mu} + \frac{1}{2} (1 - \zeta_i^\mu) \log \frac{1 - \zeta_i^\mu}{1 - O_i^\mu} \right)$$

✚ Fijarse que cuando  $O_i^\mu$  se acerca a  $\zeta_i^\mu$ , entonces la función  $E()$  tiende a 0 y cuando se aleja, tiende a  $\infty$ .

# Redes neuronales artificiales

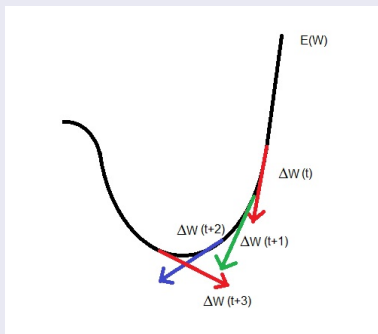
## Consideraciones acerca del método de retropropagación.

- ✚ Si se usa como función de activación la  $\tanh()$  y  $\zeta_i^\mu \in \{1, -1\}$ ,  $O_i^\mu$  se acercará mucho a  $\zeta_i^\mu$  pero nunca adoptará ese valor.
- ✚ Por otro lado, al diferenciar la función  $E()$ , el  $\delta_i^\mu$  quedará definido como
$$\delta_i^\mu = \beta(\zeta_i^\mu - O_i^\mu)$$
donde  $\beta$  corresponde a la definición de  $g(h) = \tanh(\beta h)$ .
- ✚ Fijarse que desapareció el factor  $g'(h_i^\mu)$  del cálculo de  $\delta_i^\mu$ .

# Redes neuronales artificiales

## Momentum

- Otro inconveniente que suele pasar cuando el  $W$  está cerca de un mínimo (sea local o global) es que puede quedar oscilando en torno al mínimo e incluso diverger.



# Redes neuronales artificiales

## Momentum

- Una forma de abordar este problema es el momentum:

$$\Delta w_{ij}(t+1) = -\eta \frac{\partial E}{\partial w_{ij}} + \alpha \Delta w_{ij}(t)$$

donde  $\alpha$  está entre 0 y 1,

- $\alpha = 0,8$  o  $\alpha = 0,9$  son frecuentemente usados.
- Si la función  $E()$  está en una región plana,  $\Delta w_{ij}(t)$  será similar a  $\frac{\partial E}{\partial w_{ij}}$  y por lo tanto  $\Delta w_{ij}(t+1)$  será verás incrementado en módulo.
- Si la función  $E()$  está en un valle o una región oscilatoria,  $\Delta w_{ij}(t+1)$  tenderá a compensarse para evitar oscilaciones.

# Redes neuronales artificiales

## $\eta$ adaptativo

- ✚ Si, en forma consistente,  $E()$  decrece en un momento del aprendizaje (por ejemplo, durante  $k$  iteraciones seguidas), quizás sería interesante incrementar a  $\eta$ .
- ✚ Por otro lado, si  $E()$  comienza a incrementarse en un momento del aprendizaje (por ejemplo, durante  $k'$  iteraciones seguidas) quizás sería interesante decrementar a  $\eta$  para evitar divergencia.



# Redes neuronales artificiales

## $\eta$ adaptativo

- ✦ Esto se puede formalizar adaptando el valor de  $\eta$  según
$$\Delta\eta = \begin{cases} +a & \text{si } \Delta E < 0 \text{ en forma consistente} \\ -b\eta & \text{si } \Delta E > 0 \text{ empieza a ser consistente} \\ 0 & \text{en otro caso} \end{cases}$$

# Redes neuronales artificiales

## $\eta$ adaptativo

- ✚ También es posible actualizar el  $\eta$  por neurona (para los pesos de las conexiones que llegan a esa neurona) o por conexión (un  $\eta$  por cada  $w_{ij}$ ).

# Redes neuronales artificiales

## Otros métodos de minimización

- Estas modificaciones fueron propuestas basadas en el algoritmo original donde la dirección de descenso ( $\Delta W$ ) está dada por el método del gradiente descendente.
- Sin embargo existen otras alternativas para encontrar la dirección de descenso las cuales no dependen exclusivamente del gradiente.
- Incluso, hay alternativas que no requieren que la función de costo sea derivable (o conocer la derivada de la función de costo).